Binary Coding by Matrix Classifier for Efficient Subspace Retrieval

Lei Zhou
School of Computer Science and Engineering and Beijing Advanced Innovation Center for Big Data and Brain Computing, Beihang University
Beijing, China
leizhou@buaa.edu.cn

Xiao Bai*
School of Computer Science and Engineering and Beijing Advanced Innovation Center for Big Data and Brain Computing, Beihang University
Beijing, China
baixiao@buaa.edu.cn

Xianglong Liu
School of Computer Science and Engineering, Beihang University
Beijing, China
xlliu@nlsde.buaa.edu.cn

Jun Zhou
School of Information and Communication Technology,
Griffith University
Queensland, Australia
jun.zhou@griffith.edu.au

ABSTRACT

Fast retrieval in large-scale database with high-dimensional subspaces is an important task in many applications, such as image retrieval, video retrieval and visual recognition. This can be facilitated by approximate nearest subspace (ANS) retrieval which requires effective subspace representation. Most of the existing methods for this problem represent subspace by point in the Euclidean space or the Grassmannian space before applying the approximate nearest neighbor (ANN) search. However, the efficiency of these methods can not be guaranteed because the subspace representation step can be very time consuming when coping with high dimensional data. Moreover, the transforming process for subspace to point will cause subspace structural information loss which influence the retrieval accuracy. In this paper, we present a new approach for hashing-based ANS retrieval. The proposed method learns the binary codes for given subspace set following a similarity preserving criterion. It simultaneously leverages the learned binary codes to train matrix classifiers as hash functions. This method can directly binarize a subspace without transforming it into a vector. Therefore, it can efficiently solve the large-scale and high-dimensional multimedia data retrieval problem. Experiments on face recognition and video retrieval show that our method outperforms several state-of-the-art methods in both efficiency and accuracy.

*Corresponding author (baixiao@buaa.edu.cn)

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICMR '18, June 11–14, 2018, Yokohama, Japan
© 2018 Association for Computing Machinery.
ACM ISBN 978-1-4503-5046-4/18/06…$15.00
https://doi.org/10.1145/3206025.3206058

CCS CONCEPTS

• **Information systems** → **Nearest-neighbor search**; *Image search*; *Video search*;

KEYWORDS

Large-Scale Multimedia Data Retrieval; Subspace Retrieval; Binary Coding; Hashing; Matrix Classifier

ACM Reference Format:
Lei Zhou, Xiao Bai, Xianglong Liu, and Jun Zhou. 2018. Binary Coding by Matrix Classifier for Efficient Subspace Retrieval. In *ICMR '18: 2018 International Conference on Multimedia Retrieval, June 11–14, 2018, Yokohama, Japan.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3206025.3206058

1 INTRODUCTION

Linear subspace is important for data representation in many multimedia and machine learning tasks. It has demonstrated excellent property in capturing the structural information of some specific data, e.g., face images under different illuminations [3], sequential frames in the video clips [22], classes of similar object [2]. In a high dimensional space, data can be either represented as subspaces or vectors. In practice, it is better to represent these data by subspaces rather than vector features because the intrinsic dimension of high-dimensional data is often much smaller than the ambient dimension [24].

When a query is a vectorized image representation and the dataset is composed of vectors, the problem can be solved by the nearest neighbor search. Thanks to the recent growth of multimedia contents, rapid search in a large dataset is greatly demanded. Many studies aim at efficiently retrieving relevant data from large-scale datasets. Because of the high-computational cost, traditional linear search (or exhaustive search) is not appropriate for searching in a large-scale database. Instead of linear search, many practical strategies are proposed for approximate nearest neighbor (ANN) search [7, 14, 15, 21]. The most general known approach is hashing

algorithm which can efficiently solve high-dimensional and large-scale problems [1, 11, 25, 29].

When subspace is used as the representation, how to efficiently find the approximate nearest subspace (ANS) for a given query subspace has to be studied. This problem is related to a wide range of multimedia and pattern recognition applications such as image retrieval [12, 31], video retrieval [30], face recognition [5, 19], and action recognition [23]. A well known work on ANS retrieval was proposed by Basri *et al.* [2]. A mapping was used to transform subspaces to points, then the problem is solved by well-studied ANN search on points. Motivated by this work, some recent approaches [13, 26] utilized the idea of locality sensitive hashing (LSH) [6] to accelerate the searching process. More recently, Xu *et al.* [30] proposed a video retrieval framework, which defined a similarity-preserving distance metric between an image and its orthogonal projection in the subspace of the video. These approaches, however, represent subspaces by points in the Euclidean space or Grassmannian space before applying the traditional ANN methods. A subspace with dimension D will be transformed into a vector whose size is $\Theta(D^2)$. In addition, the transforming process for subspace to point may cause subspace structural information loss and additional calculational cost. Therefore, they can not efficiently solve the ANS retrieval problem, specifically for very high-dimensional data in a large-scale dataset.

In this paper, we present an efficient method for linear subspace retrieval based on hashing algorithm. The superiority of hashing algorithm in information retrieval includes two main aspects. First, binary signature generated by hashing can significantly reduce the storage space for large-scale data. In addition, pairwise data similarity can be fast estimated by calculating the Hamming distance between the corresponding binary signatures. Motivated by these two properties, we develop a method which can efficiently binarize the subspace and simultaneously preserve the similarity between subspaces. First we learn the binary codes for given subspace set following defined similarity preserving criterion. In order to solve the out-of-sample problem (a new query which is not used in training set), our algorithm leverages the learned binary codes to train a set of matrix classifiers to predict the code for query subspace. Here each bit of the binary code for subspace is considered as a class label for that subspace. Then we train r binary classifiers for the r bits binary code. For any given out-of-sample query subspace, the corresponding r bits binary code can be efficiently generated by the r trained binary classifiers. In our method, we use the Support Matrix Machines (SMM) [18] as the binary matrix classifier due to its excellent capability in capturing the structure information within the matrix data.

Compared with the previous works [2, 13, 26, 30], the advantages of our method are that it achieves fast computation without converting a subspace into a vector in the data space, and the classifier set can generate compact hash codes efficiently. In the offline stage, we learn the binary codes for given subspace set with the similarity between subspaces and simultaneously train binary classifiers by the learned codes. All calculations in this stage are done directly on subspace matrices. In the online stage, the query subspace matrix is directly binarized by the trained classifiers. Then we can use the Hamming distance between binary codes to quickly retrieve the ANS. Thus, our method can efficiently solve the ANS retrieval problem even with a high-dimensional and large-scale subspace dataset.

The main contributions of this paper are as follows.
(1) We propose an efficient hashing based solution for the ANS retrieval problem. Our method binarizes a subspace by defined similarity preserving criterion. The compact binary code can significantly reduce the storage and computational cost.
(2) The proposed method directly binarizes the subspace by a matrix classifier without transforming the subspace matrix into the vector form. Therefore, the proposed method can represent a subspace by a shorter binary code with less subspace structural information loss.
(3) Our method can efficiently deal with large-scale and high-dimensional multimedia data retrieval problem due to the compact and discriminative binary codes. Experimental results on face recognition and video retrieval show that our method outperforms the state-of-the-art in both retrieval efficiency and accuracy.

2 RELATED WORK

In this section, we review the related works on ANS retrieval. One of the earliest method in this area was proposed by Basri, Hassner and Zelnik-Manor (BHZ) [2], which represents a subspace $\mathcal{S}$ as its orthogonal bases matrix S [1]. Then a scalable mapping is defined to map S to a vector by taking the entries of the upper triangular portion of SS^T with the diagonal entries scaled by $1/\sqrt{2}$. This allows traditional ANN vector search methods to be applicable on the data. Based on this mapping, this method can solve the ANS problem when both query and database elements may be either points or subspaces of different dimensions. However, when the dimension D of the subspace is very high, the corresponding vector has size $D(D+1)/2$, so the efficiency of this method suffers from high-dimensional data.

Motivated by the above method, Wang *et al.* provided a LSH-based solution to the ANS retrieval problem [26]. This method represents the subspace as a point on Grassmannian, which is named as GLH (Grassmannian-based Locality Hashing). The LSH takes advantage of the geometric position of different subspaces as encoded in their principal angles with random lines, so this method provides good results in high ambient dimension. However, there still exists instability due to the strong randomness of LSH which will make the retrieval accuracy decreased when the training dataset is small [6]. A similar approach proposed by Stephen and Bruce introduces a subspace forest for determining the approximate nearest neighbors of points on Grassmann manifolds [20], which can be used to rapidly classify actions by

[1]Similarly, in our paper, all operations on subspace are done on its orthogonal bases matrix.

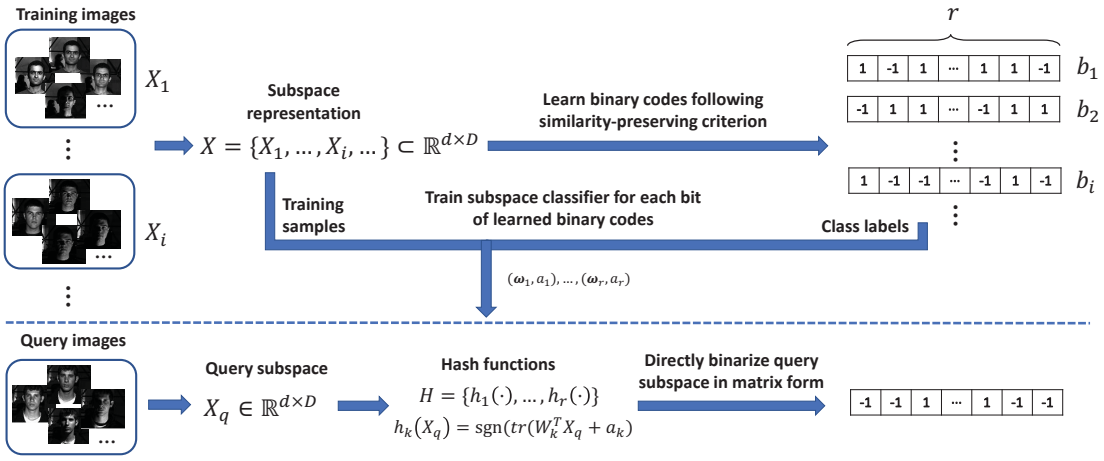


Figure 1: The proposed hashing based method for approximate nearest subspace retrieval. The process above the dashed line is the offline training stage which aims to learn a set of matrix classifiers that map subspaces to corresponding binary codes. The process under the dashed line is the online searching stage which directly binarizes the query subspace by the trained matrix classifiers. Then we can calculate the Hamming distance between binary codes to quickly retrieve the approximate nearest subspace.

a set of labeled samples. The subspace forest works in a manner conceptually similar to randomized forests for ANN computations, but respects the Grassmann manifold geometry.

Another hash-based ANS retrieval was reported by Ji *et al.*[13]. They presented a method to produce similarity preserving Binary Signatures for Subspaces (BSS). In this method, the angular similarity and angular distance between subspaces are first defined. Then a sign-random-projection function is applied to generate binary signatures for subspace. Ji *et al.* proved that the Hamming distance between the binary signatures is an unbiased estimator of the pairwise angular distance. Hence, they applied LSH to the ANN vector search problem. Since this method transforms the subspace matrix into higher dimensional vector, it needs a long binary code to preserve the similarity between subspaces.

More recently, Xu *et al.* [30] proposed a video retrieval framework, which defines a similarity-preserving distance metric between an image and its orthogonal projection in the subspace of the video. They first represented videos as subspaces of frames, and then asymmetrically projected images and videos into a common Hamming space, where they could efficiently retrieve the most relevant videos provided with an image query. Actually, their method also converts the subspace matrix into vector, and then learns similarity-preserving binary codes by the Euclidean distance between images and video subspaces.

The transforming process converting subspace to vector may cause subspace structural information loss and additional calculational cost. The problems of these methods have been solved in our algorithm. We leverage the binary classifier as hash function to encode the subspace into binary code without transforming the subspace matrix into the vector form. On one hand, the compact binary code for subspace can significantly reduce the storage space and computational cost for large-scale data. On the other hand, experiments show that our method can represent a subspace by very short binary code with less information loss.

3 PROPOSED METHOD

In this section, we describe the proposed hashing based method that solves the ANS retrieval problem. We first define the similarity between subspaces and construct an affinity matrix for learning the binary codes. Then the binary codes for given subspace set can be learned by the similarity preserving criterion. In order to solve the out-of-sample problem, our algorithm leverages the learned binary codes to train a set of binary classifiers as hash functions. Here each bit of the binary code for subspace is considered as a class label for that subspace. Then we can train r binary classifiers for the r bits binary code. For any given out-of-sample query subspace, the corresponding r bits binary code can be efficiently generated by the r trained binary classifiers. The process of the proposed method is shown in Figure 1.

3.1 Similarity and Affinity Matrix between Subspaces

First we define the similarity between linear subspaces. In our method, we use the definition of principal angles proposed in [10] which reveal the relative position of two subspaces. Let $\mathcal{F}$ and $\mathcal{G}$ be linear subspaces in $\mathbb{R}^d$, whose dimensions satisfy

$$p = \dim(\mathcal{F}) \geq \dim(\mathcal{G}) = q \geq 1 \tag{1}$$

The principal angles $\{\theta_i\}_{i=1}^q \in [0, \pi/2]$ between these two subspaces are defined recursively by

$$\cos\theta_k = f_k^T g_k = \max_{\substack{f\in\mathcal{F},\|f\|_2=1 \\ f^T[f_1,...,f_{k-1}]=0}} \max_{\substack{g\in\mathcal{G},\|g\|_2=1 \\ g^T[g_1,...,g_{k-1}]=0}} f^T g \quad (2)$$

Let F be an $d \times p$ matrix whose columns are orthonormal bases for subspace $\mathcal{F}$. Let G be an $d \times q$ matrix whose columns are orthonormal bases for subspace $\mathcal{G}$. It is easy to prove that the cosine of each principal angles equals a singular value of $F^T G$. Assume that the SVD of $F^T G$ is

$$F^T G = U diag(\sigma_1, \sigma_2, ..., \sigma_q) V^T \quad (3)$$

Where $1 \geq \sigma_1 \geq \sigma_2 \geq ... \geq \sigma_q \geq 0$. Assume that the principal angles satisfy $0 \leq \theta_1 \leq \theta_2 \leq ... \leq \theta_q \leq \pi/2$, we can get

$$\cos\theta_i = \sigma_i, \quad i = 1, 2, ..., q \quad (4)$$

Then the similarity[2] which reflects the angle similarity between subspaces $\mathcal{F}$ and $\mathcal{G}$ can be defined as

$$S_{\mathcal{F},\mathcal{G}} = \cos\theta_{\mathcal{F},\mathcal{G}} = \frac{\sum_{i=1}^q \cos^2\theta_i}{\sqrt{p}\sqrt{q}} = \frac{\sum_{i=1}^q \sigma_i^2}{\sqrt{p}\sqrt{q}} \quad (5)$$

With the property of singular value that $\sum_{i=1}^q \sigma_i^2 = \|F^T G\|_F^2$, where $\|A\|_F$ is the Frobenius norm of A, we can rewrite the similarity between subspaces as

$$S_{\mathcal{F},\mathcal{G}} = \frac{\|F^T G\|_F^2}{\sqrt{p}\sqrt{q}} \quad (6)$$

Given the defined similarity between subspaces and $S \in [0, 1]$, we can construct an $n \times n$ affinity matrix S for a subspace set $\{\mathcal{X}_1, \mathcal{X}_2, ..., \mathcal{X}_n\}$. Assume that $\{X_1, X_2, ..., X_n\}$ are matrices whose columns are orthonormal bases for $\{\mathcal{X}_1, ..., \mathcal{X}_n\}$, then the affinity matrix S between subspaces can be defined as

$$S_{ij} = \begin{cases} S_{X_i,X_j} & \text{if } S_{X_i,X_j} \geq \epsilon \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

Where $\epsilon \in [0, 1]$. We select ϵ to make S a sparse matrix which shows the local similarity structure of the subspace set. The sparsity of affinity matrix not only leads to much lower storage space, but also brings a significant reduction to the computational cost of subsequent operations.

3.2 Learning Binary Codes for Subspaces

Given a set of linear subspaces $\mathcal{X} = \{\mathcal{X}_1, \mathcal{X}_2, ..., \mathcal{X}_n\} \subset \mathbb{R}^d$, the problem we address is to retrieve the approximate nearest subspace from the subspace dataset for the given query subspace. In order to rapid retrieval, we transform the subspace into binary code. Here we assume that X_i is a matrix whose columns are orthonormal bases of $\mathcal{X}_i$ and b_i is the corresponding r bit binary code for X_i. Our goal is to learn a set of hash functions which can well preserve the similarity between $\mathcal{X}$. Formally, the Hamming distance between two binary codes b_i and b_j is small when $\mathcal{X}_i$ and $\mathcal{X}_j$ is similar (S_{ij} is close to 1), the Hamming distance between b_i and b_j is large when $\mathcal{X}_i$ and $\mathcal{X}_j$ is dissimilar (S_{ij} is close to 0).

[2]Another definition in [26] is similar to Eq. (5). Here, we normalize the similarity for convenient calculation.

Based on the definition of the affinity matrix between subspaces, we first calculate the ground-truth affinity matrix S for $\mathcal{X}$. Where S shows the local similarity structure of $\mathcal{X}$. $H(\cdot)$ is a set of r hash functions we want to learn, $H(\cdot) = \{h_1(\cdot), h_2(\cdot), ..., h_r(\cdot)\}$. We learn the binary codes and hash functions simultaneously. In other words, our method aims to learn a set of binary codes $B = \{b_i\}_{i=1}^n \in \{-1, 1\}^r$ which can well preserve the similarity between $\mathcal{X}$ and simultaneously learn a set of matrix classifiers as hash functions which can map $\mathcal{X}$ to the learned binary codes.

The Hamming distance between b_i and b_j is given by the number of different bits between them. Since $\{b_i\}_{i=1}^n \in \{-1, 1\}^r$, the Hamming distance can be calculated as

$$d_{Ham}(b_i, b_j) = \frac{r - b_i^T b_j}{2} \quad (8)$$

To meet the similarity preserving objective, the optimization objective can be written as

$$\min_{B\in\{-1,1\}^{r\times n}} \sum_{i=1}^n \sum_{j=1}^n S_{ij} \frac{r - b_i^T b_j}{2} \quad (9)$$

Here $\sum_{i=1}^n \sum_{j=1}^n S_{ij} r$ is constant and $b_i^T b_i = r$. Let L be a diagonal matrix given by $L_{ii} = \sum_{j=1}^n S_{ij}$, the objective function Eq. (9) is equal to

$$\min_{B\in\{-1,1\}^{r\times n}} tr(B^T(L - S)B) \quad (10)$$

It is easy to find that except the constraint $B \in \{-1, 1\}^{r\times n}$, the above objective Eq. (10) forms a Laplacian Eigenmap problem. So we relax this constraint but still keep the similarity preserving objective. Then we can get the following Laplacian Eigenmap problem

$$\min_{\bar{B}} \quad tr(\bar{B}^T(L - S)\bar{B}) \quad (11)$$

$$\text{subject to } \bar{B}^T L \bar{B} = I, \bar{B}^T L 1 = 0$$

The solution of this Laplacian Eigenmap problem is the r eigenvectors corresponding to the r minimal non-zero eigenvalues of $L - S$. Assume that the solution is given by $\bar{B} = \{\bar{b}_i\}_{i=1}^n \in \{-1, 1\}^r$ whose columns are the r eigenvectors, they are not binary codes but real-valued vectors.

Next we need to transform $\bar{B}$ into binary codes via a r-dimensional thresholding vector. As mentioned in Spectral Hashing [27], a good binary code should satisfy that each bit has 50% chance of being 1 or -1, and different bits are independent of each other. Therefore, we respectively select the median value of the r eigenvectors as a thresholding vector $v = (v_1, v_2, ..., v_r)$. If the l-th element of $\bar{b}_i$ is larger than v_l, $b_i^{(l)} = 1$ ($b_i^{(l)}$ is the l-th bit of b_i). Otherwise, $b_i^{(l)} = -1$. Furthermore, as the solution eigenvectors of Eq. (11) are orthogonal to each other, different bits $\{b_i^{(1)}, b_i^{(2)}, ..., b_i^{(r)}\}$ in the learned binary codes will be independent of each other. Then we get discriminative binary codes B which can well preserve the similarity between subspaces $\mathcal{X}$.

3.3 Training Matrix Classifiers

Although we have learned the binary codes for subspace set $\mathcal{X}$, it does not mean that the ANS retrieval problem has been solved. For a given new query subspace, if we add it into the subspace set $\mathcal{X}$ and update all binary codes according to the method we present above, the high computational cost obviously can not be accepted for practical application. According to the idea of hashing algorithm, we need a set of hash functions which can map the query subspace to corresponding binary code.

Since the subspace is generally represented as matrix composed by its orthonormal bases, we leverage the matrix classifier as hash function. Let each bit of the binary code for subspace can be considered as a class label (class '1' or class '−1') for that subspace. A binary matrix classifier can be trained for each bit of the learned binary codes. Then we use the r binary matrix classifiers to obtain the r-bit binary code for any query subspace.

In this paper, we use the Support Matrix Machine (SMM) [18] as the matrix classifier model due to its excellent ability in capturing the structure information within the matrix data. To capture the correlation among columns or rows within the matrix data, SMM imposes a low-rank constraint on regression matrix W to leverage the structure information. Since the nuclear norm $\|W\|_*$ is the best convex approximation of rank(W) over the unit ball of matrices, and the hinge loss is well fit for large margin principle, the formulation of SMM model is

$$\min_{W,a} \frac{1}{2}\|W\|_F^2 + \tau\|W\|_* + C\sum_{i=1}^{n}\{1 - y_i[tr(W^T X_i) + a]\}_+ \quad (12)$$

Where $\{1-e\}_+ = \max\{0, 1-e\}$ is the hinge loss function, W is the matrix of regression coefficients, a is an offset term, C is a regularization parameter, and y_i is the class label of X_i. SMM is based on a penalty function which is the combination of the squared Frobenius norm $\|W\|_F^2$ and the nuclear norm $\|W\|_*$.

Because each bit of the binary code is independent of each other, the binary classifiers of each bit can be trained independently. For the bit $b_i^{(k)}$, the corresponding SMM can be trained by following formulation

$$\min_{W_k,a_k} \frac{1}{2}\|W_k\|_F^2 + \tau\|W_k\|_* + C\sum_{i=1}^{n}\{1 - b_i^{(k)}[tr(W_k^T X_i) + a_k]\}_+ \quad (13)$$

Here we consider $b_i^{(k)}$ as a class label for X_i. After learning the binary classifier, the corresponding hash function can be formulated as $h_k(X) = sgn(tr(W_k^T X) + a_k)$.

Since the objective function Eq. (13) is convex in both W and a, we use the Alternating Direction Method of Multipliers (ADMM) [9] to optimize this convex problem. The objective function Eq. (13) can be equivalently written as

$$\min_{W_k,a_k,V_k} H(W_k, a_k) + G(V_k) \quad (14)$$

$$\text{subject to} \quad V_k - W_k = 0$$

Where

$$H(W_k, a_k) = \frac{1}{2}\|W_k\|_F^2 + C\sum_{i=1}^{n}\{1 - b_i^{(k)}[tr(W_k^T X_i) + a_k]\}_+$$

and

$$G(V_k) = \tau\|W_k\|_*$$

Then the augmented Lagrangian function of Eq. (14) is

$$\begin{aligned} L(W_k, a_k, V_k, \Lambda) =& H(W_k, a_k) + G(V_k) + tr[\Lambda^T(V_k - W_k)] \\ &+ (\rho/2)\|V_k - W_k\|_F^2 \end{aligned} \quad (15)$$

Where $\rho > 0$ is augmented Lagrangian parameter, and Λ is penalty parameter. ADMM solves Eq. (15) with the iterative equations

$$(W_k^{(i+1)}, a_k^{(i+1)}) = \min_{W_k,a_k} L(W_k, a_k, V_k^{(i)}, \Lambda^{(i)}) \quad (16)$$

$$V_k^{(i+1)} = \min_{V_k} L(W_k^{(i+1)}, a_k^{(i+1)}, V_k, \Lambda^{(i)}) \quad (17)$$

$$\Lambda^{(i+1)} = \Lambda^{(i)} + \rho(V_k^{(i+1)} - W_k^{(i+1)}) \quad (18)$$

The condition of stopping iteration is that primal residual $r^{(i+1)} = \|V_k^{(i+1)} - W_k^{(i+1)}\|_F^2$ and dual residual $t^{(i+1)} = \rho\|V_k^{(i+1)} - V_k^{(i)}\|_F^2$ converge to 0.

From the solution of Eq. (13), we can successively obtain r hash functions $\{h_1(\cdot), h_2(\cdot), ..., h_r(\cdot)\}$, where

$$h_k(X) = sgn(tr(W_k^T X) + a_k) \quad (19)$$

Then we can efficiently get the corresponding r bits binary code for query subspace by these hash functions.

In our method, the dimension of query subspace must be equal to the training subspace due to the restriction by W_k. However, in practice, the dimension of query subspace may be different from training subspace (for example, when query is an image). Here we expand our method with the query subspace of different dimensions. Assume that the dimension of training subspace is d_t, and d_q for query subspace. In the case that $d_q < d_t$, we can add $d_t - d_q$ bases by randomly choosing from the orthogonal bases of query subspace. In the case that $d_q > d_t$, we can only select the first d_t orthogonal bases of query subspace. Experiments show that our method can be applied in query subspace of different dimensions, with little loss in retrieval accuracy.

3.4 Complexity Analysis

The whole process of proposed method is described in Algorithm 1. Here we analyse the time complexity of our algorithm. In the offline training process, if the given subspace set $X = \{X_i\}_{i=1}^n \subset \mathbb{R}^{d\times D}$, the matrix multiplication for similarity between subspaces in Eq. (6) needs $O(dD^2)$. The affinity matrix can be computed in $O(n^2dD^2)$. In the online searching process, computing the r bit binary code by Eq. (19) for query subspace costs $O(rDd)$. Therefore, the encoding time for query subspace in our algorithm is linearly dependent on the dimension d of training subspaces, regardless the size of training set. Experiments show that our method outperforms the previous works in the query time.

Table 1: The top 1 retrieval accuracy of four methods on two face databases, with different dimensions d_q of query subspaces. Here we show the accuracy of BSS with binary code length $r = 2000$, and the accuracy of our method with binary code length $r = 32$.

Method	Extended YaleB			YouTube Faces		
	$d_q = 4$	$d_q = 9$	$d_q = 13$	$d_q = 4$	$d_q = 9$	$d_q = 13$
BHZ [2]	0.8158	0.7865	0.7749	0.7318	0.7015	0.6803
GLH [26]	0.8772	0.8919	0.9001	0.7693	0.8014	0.8130
BSS [13]	0.9094	0.9653	0.9240	0.8072	0.8251	0.8062
Ours	**0.9415**	**0.9812**	**0.9561**	**0.8414**	**0.8821**	**0.8572**

Table 2: The query time (second) for one retrieval by four methods on two face databases, with different dimensions d_q of query subspaces. Here we show the query time of BSS with binary code length $r = 2000$, and the query time of our method with binary code length $r = 32$.

Method	Extended YaleB			YouTube Faces		
	$d_q = 4$	$d_q = 9$	$d_q = 13$	$d_q = 4$	$d_q = 9$	$d_q = 13$
BHZ [2]	0.3102	0.6980	1.086	0.9517	2.174	3.381
GLH [26]	0.0824	0.0824	0.1282	0.2415	0.2415	0.3757
BSS [13]	5.006e-3	5.006e-3	5.006e-3	0.0221	0.0221	0.0221
Ours	**6.257e-5**	**6.257e-5**	**6.257e-5**	**2.735e-4**	**2.735e-4**	**2.735e-4**

Algorithm 1: The proposed method for ANS retrieval.

Input: Training subspaces $X = \{X_i\}_{i=1}^n$ and the hash bits r.

Output: Hash functions $H = \{h_i\}_{i=1}^r$ and the binary codes $B = \{b_i\}_{i=1}^n$ of X.

Steps:

1. Compute similarity between subspaces by Eq. (6).
2. Construct affinity matrix S for X.
3. Optimize the Laplacian Eigenmap problem Eq. (11), the solution is $\bar{B} = \{\bar{b}_i\}_{i=1}^n$
4. Choose the r median values $v_1, v_2, ..., v_r$ of the r eigenvectors obtained in step 3 as thresholding.

for $i = 1, ..., n$ **do**
  for $l = 1, ..., r$ **do**
    If the l-th element of $\bar{b}_i$ is larger than v_l, $b_i^{(l)} = 1$.
    Otherwise, $b_i^{(l)} = -1$.
  end
end

5.Optimize Eq. (13) with B fixed.

for $k = 1, ..., r$ **do**
  Optimize Eq. (13) by ADMM algorithm, achieving W_k and a_k.
  $h_k(X) = sgn(tr(W_k^T X) + a_k)$.
end

4 EXPERIMENTS

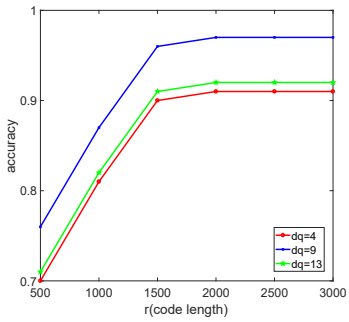
To evaluate the performance of our ANS retrieval method, we undertook two sets of experiments on three public datasets. The first experiment was face recognition with two datasets: the extended Yale Face Database B [16] (including the Yale Face Database B [8]) and the YouTube Faces Database [28]. Another was video retrieval on the Big Bang Theory (BBT) [17].

4.1 Face Recognition

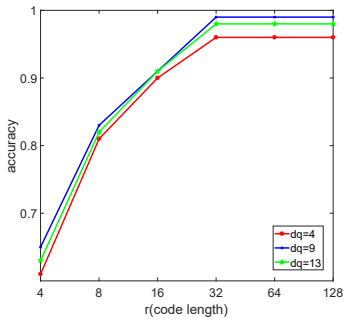
As subspace is commonly used to capture the appearance of faces under varying illuminations, we test the performance of our method by searching the approximate nearest neighbors on the extended Yale Face Database B and the YouTube Faces Database. We compared our result with several state-of-the-art methods BHZ [2], GLH [26], and BSS [13] in both query time and accuracy.

The extended Yale Face Database B (including 10 human subjects from the Yale Face Database B) contains 38 human subjects each with about 576 face images under 64 different illumination conditions and nine poses. The YouTube Faces Database contains 3425 videos of 1595 different people. For each people, there are 2.15 videos on average. We used the frame-images database which contains sequential frames extracted from the videos, and cropped only the face from each image.

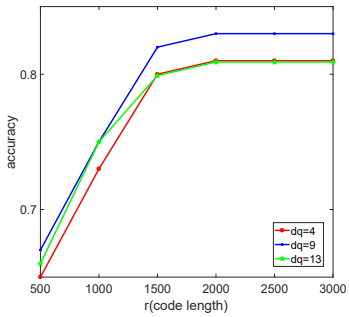
In our experiment, we first resized all the face images to 32×32. For the extended Yale Face Database B, we constructed 9-dimensional training subspaces by randomly choosing 30 images out of the 64 images for each people in one pose, vectorized them to 30 1024-dimensional vectors and computed their first 9 principal components. The rest images were used for constructing query subspaces in the same manner. In practice, the dimension of query subspace may be different from training subspace. Therefore, we constructed query subspaces with different dimensions $d_q = 4, 9, 13$ which include dimensions smaller or equal or larger than the dimension of training subspaces. Thus there



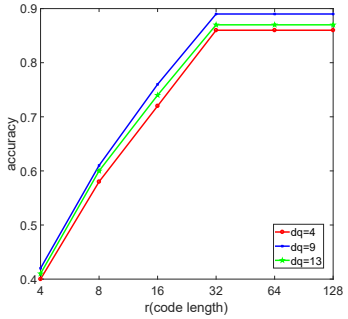
(a) BSS [13] on YaleB



(b) Our method on YaleB



(c) BSS [13] on Youtube Faces



(d) Our method on Youtube Faces

Figure 2: (a) The accuracy of BSS method with different code length on the extended Yale Face Database B. (b) The accuracy of our method with different code length on the extended Yale Face Database B. (c) The accuracy of BSS method with different code length on the YouTube Faces Database. (d) The accuracy of our method with different code length on the YouTube Faces Database.

were $38 \times 9 = 342$ training subspaces and $38 \times 9 \times 3 = 1026$ query subspaces (three different dimensions). In the training process, we set $\epsilon = 0.6$ in Eq. (7). For the YouTube Faces Database, we randomly chose 30 frame-images from each video as the training set, and the rest images were used for the query set. Then we constructed training subspaces and query subspaces by the same manner used in the extended Yale Face Database B. In the training process, we set $\epsilon = 0.6$. For each query subspace, we first encoded it by the learned hash functions Eq. (19). Then we retrieved for the nearest subspace in the training set in terms of Hamming distance between the binary codes. The top 1 retrieval accuracy of our method, compared with BHZ [2], GLH [26] and BSS [13], are shown in Table 1. Here we show the accuracy of BSS with binary code length $r = 2000$, and the accuracy of our method with binary code length $r = 32$. Due to the strong ability of classifiers to express the mapping of subspaces and binary codes, our method achieves better accuracy than previous work with shorter code.

As BSS also produces binary codes for subspaces, we compared our method with BSS by using different code lengths on the extended Yale Face Database B. As shown in Figure 2, when $r = 32$, our method achieves an accuracy of nearly 100%. BSS needs more than 1000 bit binary codes to achieve a good result. In our experiment, BHZ stores subspace with a vector in $1024 \times (1024 + 1)/2 = 524800$ dimensions, GLH stores subspace in 1024 dimensions, BSS stores subspace with 2000 bit binary code, and our method stores subspace with only 32 bit binary code. Thus our method has superiority in both query time and storage cost. Table 2 and Table 3 show the query time and storage cost respectively, where the query time includes both encoding time and retrieval time. We can see from Table 2 that our method is about 80 times faster than the state-of-the-art in query time due to the shorter code length.

Table 3: The storage cost (byte) of two face databases. Here we show the storage cost of BSS with binary code length $r = 2000$, and the storage cost of our method with binary code length $r = 32$.

Method	Extended YaleB	YouTube Face
Original subspace	6.981e7	6.290e9
BSS [13]	3.561e5	3.209e7
Ours	**7.596e3**	**6.846e5**

4.2 Video Retrieval

Subspace is also a good representation of video clips. In this section, we show the superiority of our method by a video retrieval experiment on the Big Bang Theory (BBT). We compared our method with state-of-the-art methods HER [17], and IBC, BBC [30].

The Big Bang Theory (BBT) is a sitcom (about 20 minutes each episode) which includes many full-view shots of about 5 to 8 characters at a time. BBT1 (The Big Bang Theory Season 1) consists of 3,341 face videos of the first 6 episodes which are represented by block Discrete Cosine Transformation (DCT) feature as used in [4], which forms a 240×240 covariance video representation. For the training process, we set $\epsilon = 0.5$ in Eq. (7).

We constructed two sets of experiments, query by image and query by video. For query by image experiment, the experiment set of HER, IBC, and BBC are as done in [17] and [30]. For our method, we randomly extracted 300 videos for training and 100 images from the rest as query for the retrieval task. We first represent training videos as subspaces and learn the corresponding binary codes. Then we train a set of classifiers as hash functions. The binary codes of query images can be efficiently obtained by the learned classifiers, where an image is considered as one-dimensional subspace. The retrieval mAP with different length of hash code is shown in Table 4. Our proposed method outperforms the state-of-the-art methods. In addition, the query time of our method is also faster than previous methods.

Table 4: Comparison with the state-of-the-art methods on BBT1 database with different length of hash code.

Method	Query-by-image mAP			
	16-bit	32-bit	64-bit	128-bit
HER [17]	0.5049	0.5227	0.5490	0.5539
IBC [30]	0.5152	0.5369	0.5561	0.5645
BBC [30]	0.5080	0.5401	0.5643	0.5718
Ours	**0.5520**	**0.5915**	**0.6014**	**0.6083**

Although HER, IBC, and BBC are designed for query-by-image video retrieval, they can also be applied to query-by-video video retrieval. We use their definition of distance between projected image and video subspace to calculate the distance between video subspaces, and then apply their coding method. For our method, we first represent training and query videos as subspaces and then use our ANS retrieval algorithm. For query by video experiment, we randomly extracted 300 videos for training and 100 videos from the rest as query. Table 5 shows the result that our method significantly performs better than the state-of-the-art methods.

4.3 Parameter Analysis

In our method, we need set suitable parameter ϵ in Eq. (7). Here, we report experimental results on different databases to illustrate the sensitivity of our methods to parameter variations. Figure 3 shows the recognition and retrieval accuracy of our method with different $\epsilon \in \{0, 0.1, 0.2, ..., 0.8, 0.9\}$. We can see that when $\epsilon = 0.5$, our method achieves best result for face recognition. When $\epsilon = 0.6$, the video retrieval accuracy reaches the highest.

Table 5: Comparison with the state-of-the-art methods on BBT1 database with different length of hash code.

Method	Query-by-video mAP			
	16-bit	32-bit	64-bit	128-bit
HER [17]	0.5214	0.5431	0.5614	0.5854
IBC [30]	0.5425	0.5542	0.5736	0.5904
BBC [30]	0.5378	0.5537	0.5752	0.5968
Ours	**0.6013**	**0.6415**	**0.6523**	**0.6595**

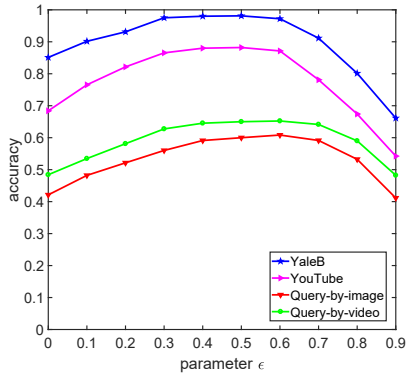


Figure 3: The accuracy of our methods with different parameter ϵ.

5 CONCLUSION

We have presented a hashing based method to solve the ANS retrieval problem. This method first learn the binary codes for training subspaces following the similarity preserving criterion. Then the learned binary codes are used to train a set of matrix binary classifiers as hash functions. The compact binary code for query subspace can be efficiently generated by these matrix classifiers. Moreover, our method is further expanded to cope with the case that query subspaces have different dimensions. Several experiments on face datasets and video dataset show that our method can achieve a significant improvement in query time and storage cost with a competitive accuracy, when compared with the state-of-the-art approaches. This suggests that our method is potentially more suitable than previous work on large-scale multimedia retrieval task for real world applications.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China project no. 61772057, in part by Beijing Natural Science Foundation project no. 4162037, and the support funding from State Key Lab. of Software Development Environment.

REFERENCES

[1] Xiao Bai, Haichuan Yang, Jun Zhou, Peng Ren, and Jian Cheng. 2014. Data-Dependent Hashing Based on p-Stable Distribution. *IEEE Transactions on Image Processing* 23, 12 (2014), 5033–5046.

[2] Ronen Basri, Tal Hassner, and Lihi Zelnikmanor. 2011. Approximate Nearest Subspace Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 2 (2011), 266–278.

[3] Ronen Basri and David W Jacobs. 2003. Lambertian reflectance and linear subspaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 25, 2 (2003), 218–233.

[4] Martin Bauml, Makarand Tapaswi, and Rainer Stiefelhagen. 2013. Semi-supervised Learning with Constraints for Person Identification in Multimedia Data. In *Computer Vision and Pattern Recognition*. 3602–3609.

[5] Xinyuan Cai, Chunheng Wang, Baihua Xiao, Xue Chen, and Ji Zhou. 2012. Deep nonlinear metric learning with independent subspace analysis for face verification. In *Proceedings of the 2012 ACM International Conference on Multimedia*. ACM, 749–752.

[6] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Symposium on Computational Geometry*. 253–262.

[7] Simone Ercoli, Marco Bertini, and Alberto Del Bimbo. 2017. Compact Hash Codes for Efficient Visual Descriptors Retrieval in Large Scale Databases. *IEEE Transactions on Multimedia* 19, 11 (2017), 2521–2532.

[8] Athinodoros S Georghiades, Peter N Belhumeur, and David Kriegman. 2001. From few to many: illumination cone models for face recognition under variable lighting and pose. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 23, 6 (2001), 643–660.

[9] Tom Goldstein, Donoghue, and Simon Setzer. 2014. Fast Alternating Direction Optimization Methods. *Siam Journal on Imaging Sciences* 7, 3 (2014).

[10] Gene H Golub and Charles F Van Loan. 1996. Matrix computations. 1996. *Johns Hopkins University, Press, Baltimore, MD, USA* (1996), 374–426.

[11] Yunchao Gong and Svetlana Lazebnik. 2013. Iterative quantization: A procrustean approach to learning binary codes. In *Computer Vision and Pattern Recognition*. 817–824.

[12] Xiaofei He. 2004. Incremental semi-supervised subspace learning for image retrieval. In *Proceedings of the 2004 ACM International Conference on Multimedia*. ACM, 2–8.

[13] J. Ji, J. Li, Q. Tian, S. Yan, and B. Zhang. 2015. Angular-Similarity-Preserving Binary Signatures for Linear Subspaces. *IEEE Transactions on Image Processing* 24, 11 (2015), 4372–80.

[14] Yu-Gang Jiang, Jun Wang, Xiangyang Xue, and Shih-Fu Chang. 2013. Query-adaptive image search with hash codes. *IEEE Transactions on Multimedia* 15, 2 (2013), 442–453.

[15] Yin-Hsi Kuo, Kuan-Ting Chen, Chien-Hsing Chiang, and Winston H Hsu. 2009. Query expansion for hash-based image object retrieval. In *Proceedings of the 2009 ACM International Conference on Multimedia*. ACM, 65–74.

[16] Kuangchih Lee, Jeffrey Ho, and David Kriegman. 2005. Acquiring linear subspaces for face recognition under variable lighting. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 27, 5 (2005), 684–698.

[17] Yan Li, Ruiping Wang, Zhiwu Huang, Shiguang Shan, and Xilin Chen. 2015. Face video retrieval with image query via hashing across Euclidean space and Riemannian manifold. In *Computer Vision and Pattern Recognition*. 4758–4767.

[18] Luo Luo, Yubo Xie, Zhihua Zhang, and Wujun Li. 2015. Support Matrix Machines. *International Conference on Machine Learning* (2015), 938–947.

[19] Nam Nguyen, Wanquan Liu, and Svetha Venkatesh. 2007. Random subspace two-dimensional PCA for face recognition. In *Pacific-Rim Conference on Multimedia*. Springer, 655–664.

[20] S O'Hara and B. A Draper. 2012. Scalable action recognition with a subspace forest. In *Computer Vision and Pattern Recognition*. 1210–1217.

[21] Fumin Shen, Wei Liu, Shaoting Zhang, Yang Yang, and Heng Tao Shen. 2015. Learning Binary Codes for Maximum Inner Product Search. In *International Conference on Computer Vision*. 4148–4156.

[22] Mei-Ling Shyu, Zongxing Xie, Min Chen, and Shu-Ching Chen. 2008. Video semantic event/concept detection using a subspace-based multimedia data mining framework. *IEEE Transactions on Multimedia* 10, 2 (2008), 252–259.

[23] Litian Sun and Kiyoharu Aizawa. 2013. Action recognition using invariant features under unexampled viewing conditions. In *Proceedings of the 2013 ACM International Conference on Multimedia*. ACM, 389–392.

[24] René Vidal. 2011. Subspace clustering. *IEEE Signal Processing Magazine* 28, 2 (2011), 52–68.

[25] Daixin Wang, Peng Cui, Mingdong Ou, and Wenwu Zhu. 2015. Learning compact hash codes for multimodal representations using orthogonal deep structure. *IEEE Transactions on Multimedia* 17, 9 (2015), 1404–1416.

[26] Xu Wang, Stefan Atev, John Wright, and Gilad Lerman. 2013. Fast Subspace Search via Grassmannian Based Hashing. In *International Conference on Computer Vision*. 2776–2783.

[27] Yair Weiss, Antonio Torralba, and Robert Fergus. 2008. Spectral Hashing. In *Advances in Neural Information Processing Systems*. 1753–1760.

[28] L. Wolf, T. Hassner, and I. Maoz. 2011. Face recognition in unconstrained videos with matched background similarity. In *Computer Vision and Pattern Recognition*. 529–534.

[29] Dayan Wu, Zheng Lin, Bo Li, Mingzhen Ye, and Weiping Wang. 2017. Deep Supervised Hashing for Multi-Label and Large-Scale Image Retrieval. In *Proceedings of the 2017 ACM International Conference on Multimedia Retrieval*. ACM, 150–158.

[30] Ruicong Xu, Yang Yang, Fumin Shen, Ning Xie, and Heng Tao Shen. 2017. Efficient Binary Coding for Subspace-based Query-by-Image Video Retrieval. In *Proceedings of the 2017 ACM International Conference on Multimedia*. ACM, 1354–1362.

[31] Mingmin Zhen, Wenmin Wang, and Ronggang Wang. 2016. Regional Subspace Projection Coding for Image Retrieval. In *Proceedings of the 2016 ACM International Conference on Multimedia Retrieval*. ACM, 205–212.